

Assignment 1: SDLC Overview - Create a one-page infographic that outlines the SDLC phases (Requirements, Design, Implementation, Testing, Deployment), highlighting the importance of each phase and how they interconnect.

Software Requirements Specification

for

Netstat Utility

Version 0.1

Prepared by

Saikat Sheet

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Intended Audience and Reading Suggestions
- 1.3 Project Scope

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Features
- 2.3 Operating Environment
- 2.4 Design and Implementation Constraints

3. System Features

- 3.1 System Feature 1
- 3.2 System Feature 2

4. External Interface Requirements

4.1 User Interfaces

4.2 Software Interfaces

4.3 Communication Interfaces

5. Other Non-functional Requirements

5.1 Performance Requirements

5.2 Safety and Reliability

5.3 Software Quality Attributes

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for the "Netstat Utility" client-server application. It outlines the networking subsystem features, client-server interactions, and logging mechanisms.

1.2 Intended Audience and Reading Suggestions

This document is intended for developers implementing the socket programming, testers verifying the netstat command outputs, and project managers overseeing the SDLC.

1.3 Project Scope

The scope involves developing a Linux-based C++ application consisting of a concurrent server and a menu-driven client. The system will retrieve networking data (TCP/UDP connections, routing tables, stats) from the Linux kernel and display it to the user.

2. Overall Description

2.1 Product Perspective

This utility mimics the standard Linux netstat tool. It operates in a distributed environment where a central server executes system-level commands and streams the output to multiple remote clients via TCP sockets.

2.2 Product Features

- Menu-Driven Interface: User-friendly command selection.
- Protocol Statistics: High-level summary of network traffic.
- Connection Filtering: Capability to filter by protocol (TCP/UDP) or application (Firefox/Chrome).
- Concurrent Handling: Server uses multi-threading/forking to handle multiple clients simultaneously.

2.3 Operating Environment

- OS: Linux (Ubuntu/CentOS recommended).
- Communication: TCP/IP Protocol Suite.
- Compiler: GCC/G++ with support for socket libraries.

2.4 Design and Implementation Constraints

- Socket Closing: Must implement a graceful shutdown to prevent "Address already in use" errors.
- Standard Utilities: Must rely on Linux system calls or the popen() pipe to fetch real-time netstat data.

3. System Features

3.1 System Feature 1: Protocol Filtering (TCP/UDP)

The client can request only TCP (-tn) or only UDP (-un) connections. The server parses the request, executes the specific command, and returns the formatted string to the client.

3.2 System Feature 2: Active Application Monitoring

The system must support specific string filtering (grep) to identify which connections belong to specific processes like web browsers.

4. External Interface Requirements

4.1 User Interfaces

A Command Line Interface (CLI) presenting a numbered menu (1-6) for users to select their desired network query.

4.2 Software Interfaces

- Linux Kernel: To access /proc/net or call system netstat.
- Socket API: For low-level network communication.

4.3 Communications Interfaces

The application will use TCP Sockets on a specific port (e.g., 9090). It must handle data serialization to ensure the table format remains readable when sent over the wire.

5. Other Non-functional Requirements

5.1 Performance Requirements

- Latency: The server should respond to client requests within 200ms.
- Concurrency: Support at least 5-10 simultaneous client connections without performance degradation.

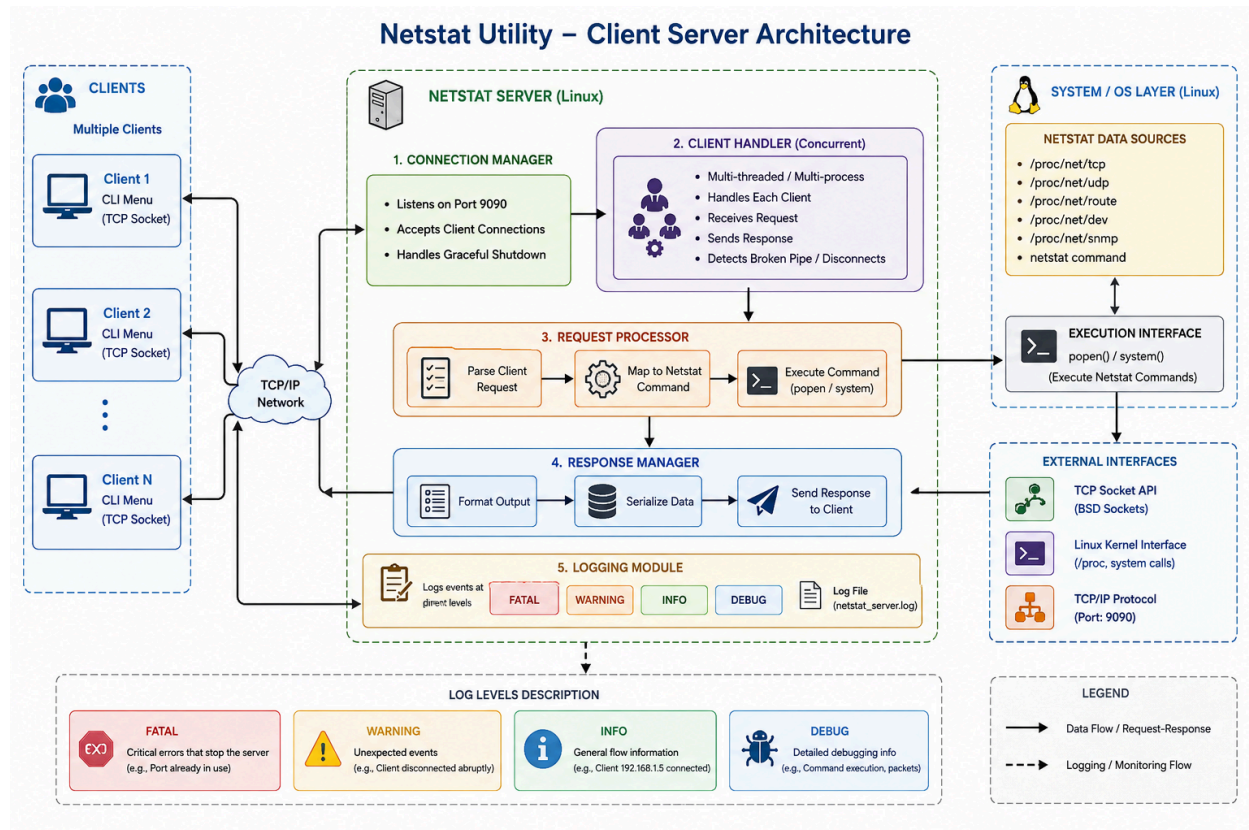
5.2 Safety and Reliability

- Socket Management: Ensure that even if a client crashes, the server detects the broken pipe and releases the associated memory and file descriptors.

5.3 Software Quality Attributes: Logging

The system must implement a robust logging mechanism with four distinct levels:

1. FATAL: For critical errors that stop the server (e.g., Port already in use).
2. WARNING: For unexpected events (e.g., Client disconnected abruptly).
3. INFO: For general flow (e.g., "Client 192.168.1.5 connected").
4. DEBUG: Detailed packet-level info or command execution strings for troubleshooting.



SDLC Overview: Developing the Netstat Utility

This infographic outlines the System Development Life Cycle (SDLC) for a client-server networking tool designed to mimic the netstat utility.

1. Requirements Phase

- **Importance:** Establishes the project scope and ensures the final product meets user needs.
- **Netstat Context:** Defining functional requirements such as the menu-driven client interface and specific command outputs (e.g., -s for statistics, -r for routing tables).
- **Interconnection:** Feeds directly into the Design phase by defining *what* needs to be built.

2. Design Phase

- Importance: Creates the technical blueprint, including system architecture and data flow.
- Netstat Context: Designing a Concurrent Server architecture to handle multiple simultaneous client requests and specifying the communication protocol between client and server.
- Interconnection: Translates user requirements into a technical map for developers to follow during implementation.

3. Implementation (Coding) Phase

- Importance: The actual construction of the software based on design specifications.
- Netstat Context: Writing C++/Linux socket code for the client-server connection, implementing the six specific netstat command filters, and integrating 4-level debug logging (FATAL, INFO, WARNING, DEBUG).
- Interconnection: Turns theoretical designs into a functional program ready for rigorous evaluation.

4. Testing Phase

- Importance: Identifies bugs and ensures the software is reliable and secure.
- Netstat Context: Verifying that netstat -un correctly filters UDP connections and ensures the server properly closes sockets after a client disconnects to prevent resource leaks.
- Interconnection: Provides a feedback loop to the Implementation phase to fix errors before the software goes live.

5. Deployment Phase

- Importance: Releasing the product to the end-user environment.
- Netstat Context: Deploying the server on a Linux host and distributing the client utility to users for real-time network monitoring.
- Interconnection: The final stage where requirements are officially fulfilled, often leading back to a new "Requirements" phase for future updates or maintenance.

Functional Breakdown: The Netstat Utility

Based on your requirements, the utility must handle specific networking subsystem information via a menu-driven interface:

System Architecture Principles

- **Concurrency:** The server must use a concurrent model (e.g., multi-threading or forking) to serve multiple clients at once without blocking.
- **Reliability:** Proper socket closure on disconnect is mandatory to ensure server stability.
- **Logging:** Adhering to standard log levels ensures the system is maintainable:
 - **FATAL:** System-wide crashes.
 - **WARNING:** Unexpected behavior that doesn't stop the app.
 - **INFO:** General operational confirmation.
 - **DEBUG:** Detailed diagnostic info for developers.

Assignment 2: Requirements Gathering - Conduct a 30-minute mock interview to gather requirements for a fictional app that helps organize community events. Summarize the requirements and how you would document and trace them in a one-page brief.

> This brief outlines the requirements gathered for "**CommuniSync**," a fictional application designed to streamline the organization, discovery, and management of local community events.

1. Executive Summary: Project "CommuniSync"

Problem Statement: Community organizers currently use fragmented tools (Facebook, spreadsheets, WhatsApp) which lead to low engagement and logistical confusion.

Solution: A centralized platform that manages the full event lifecycle—from initial planning and volunteer recruitment to attendee registration and post-event feedback.

2. Summarized Functional Requirements

Requirement ID	Requirement Description	Priority
FR-01	User Roles: System must support three roles: Organizer, Volunteer, and Attendee.	High
FR-02	Event Creation: Organizers must be able to create events with location, date, time, and capacity limits.	High
FR-03	Volunteer Management: Organizers can post specific tasks; users can sign up for "slots" (e.g., "Setup," "First Aid").	Medium
FR-04	Discovery Engine: Attendees can filter events by category (e.g., Sports, Arts) and distance (via GPS).	High

FR-05	Real-time Notifications: Automated alerts for event changes, reminders, or urgent updates via push notifications.	Medium
FR-06	Check-in System: QR-code based check-in for attendees at the event location.	Low

3. Non-Functional Requirements

- **Scalability:** The app must support up to 10,000 concurrent users during peak registration windows.
- **Usability:** Interface must meet WCAG 2.1 accessibility standards to ensure all community members can participate.
- **Security:** Personal data must be encrypted in transit and at rest, following GDPR/CCPA guidelines.

4. Documentation & Traceability Strategy

To ensure the project stays on track and every feature serves a business goal, I will use a **Requirement Traceability Matrix (RTM)**.

A. Documentation Method

- **User Stories:** I will document requirements from the user's perspective (e.g., *"As an Organizer, I want to limit the number of attendees so that I don't violate fire safety codes."*).
- **SRS (Software Requirements Specification):** A formal document containing the technical details, constraints, and data models.

B. Traceability Process

We will map each requirement through the lifecycle using a digital tool (like Jira or Azure DevOps):

1. **Business Goal → User Story:** Every feature must link back to a business objective (e.g., "Increase Community Engagement").
2. **User Story → Technical Task:** Developers link their code commits to specific Requirement IDs.
3. **Requirement → Test Case:** Every requirement must have a corresponding test case to prove it works before deployment.

5. SDLC Interconnectivity

In this project, the **Design** phase will translate the "Discovery Engine" (FR-04) into a map-view UI. The **Implementation** phase will build the GPS logic. Finally, the **Testing** phase will simulate 10,000 users to verify the **Scalability** non-functional requirement. If a test fails, we trace it back through the RTM to find exactly which requirement or design blueprint needs adjustment.

Task 1: Use-Case Diagram Creation

Objective: Create a simple use-case diagram for a Library Management System.

Steps to Perform:

- Identify at least two actors who will interact with the system (e.g., Librarian, Member).
- Determine two to three main functionalities (use cases) that the actors will perform (e.g., Check Out Book, Return Book).
- Draw a use-case diagram with these actors and use cases, showing the interactions between the actors and the system.

